

1. TITLE

Develop Dynamic Webpage Using DOM & JS

2. AIM

To develop a dynamic webpage using JavaScript and the Document Object Model (DOM) to add and delete tasks.

3. OBJECTIVES

1. To understand the Document Object Model.
2. To select and modify HTML elements using JavaScript.
3. To handle user events such as button clicks.
4. To create new DOM elements dynamically.
5. To remove elements from the webpage using JavaScript.

4. THEORY

The DOM represents an HTML document as a tree of objects. JavaScript can access these objects, change their content and attributes, create new nodes and remove existing nodes. Event listeners connect user actions such as clicks to these DOM operations.

5. ALGORITHM/PROCEDURE

1. Create a webpage containing an input field, button and task list.
2. Select the input and list elements using DOM methods.
3. Attach a click event listener to the Add Task button.
4. Read the task text from the input field.
5. Create a new list item and delete button dynamically.
6. Append the new task to the task list.
7. Attach a delete event to remove the selected task.
8. Open the page and verify dynamic interaction.

6. PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Task Manager</title>
<style>
body{font-family:Arial;background:#eef2f7;padding:35px}
.app{max-width:650px;margin:auto;background:white;padding:30px;border-radius:14px}
input{padding:11px;width:70%}
button{padding:10px 14px;margin-left:6px}
li{margin:15px 0;font-size:18px}
.delete{margin-left:12px}
</style>
</head>
<body>
<div class="app">
<h1>Dynamic Task Manager</h1>
<input id="taskInput" placeholder="Enter a task">
<button id="addBtn">Add Task</button>
<ul id="taskList">
<li>Complete DOM experiment <button class="delete">Delete</button></li>
<li>Practice JavaScript events <button class="delete">Delete</button></li>
</ul>
</div>
<script>
const input = document.getElementById("taskInput");
const addBtn = document.getElementById("addBtn");
const list = document.getElementById("taskList");

function addTask(){
  const text = input.value.trim();
  if(text === "") return;

  const item = document.createElement("li");
  item.textContent = text;

  const del = document.createElement("button");
  del.textContent = "Delete";
  del.className = "delete";

  del.addEventListener("click", () => item.remove());
  item.appendChild(del);
  list.appendChild(item);
  input.value = "";
}

addBtn.addEventListener("click", addTask);

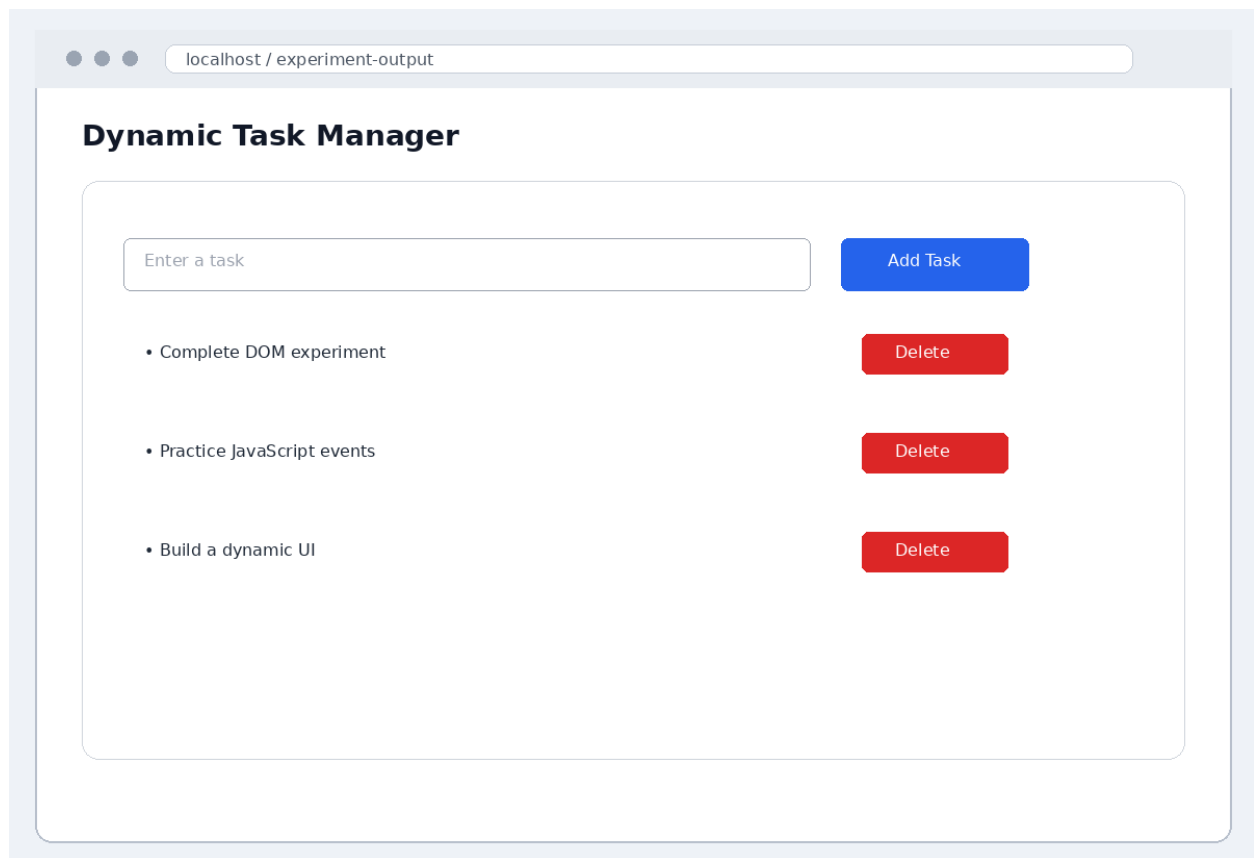
document.querySelectorAll(".delete").forEach(button => {
  button.addEventListener("click", () => button.parentElement.remove());
});
</script>
</body>
</html>
```

7. CODE EXPLANATION

Component	Purpose
getElementById()	Selects an HTML element by its id.
addEventListener()	Registers a function to respond to a user event.
createElement()	Creates a new DOM element dynamically.
textContent	Sets the visible text of a DOM element.
appendChild()	Adds a new node to the document tree.
remove()	Removes the selected task from the DOM.
trim()	Removes extra whitespace before validating input.

8. OUTPUT

Browser Output (Screenshot):



9. RESULT

Thus, a dynamic task-management webpage was successfully developed using JavaScript and DOM methods. Tasks can be added dynamically and individual tasks can be deleted through event handling.